

Bento Grid 響應式排版網站 - 實戰教學手冊

這份手冊將帶你一步步從零開始，使用 **React (Vite)** 搭配 **Tailwind CSS**，建立一個現代化、動態且極具質感的 Bento Grid（便當盒網格）網站。

步驟一：建立專案環境

首先，我們需要建立一個新的 React 專案，並安裝必要的依賴套件。

1. 打開你的終端機 (Terminal) 或命令提示字元。
2. 進入你想放置專案的資料夾，例如我們現在的桌面 Bento-grid 目錄：

```
cd c:\Users\1411\Desktop\Bento-grid
```

3. 建立一個名為 bento-grid-app 的新專案：

```
npm create vite@latest bento-grid-app -- --template react
```

4. 進入專案資料夾並安裝基本套件：

```
cd bento-grid-app
npm install
```

5. 安裝 Tailwind CSS 與其相關套件：

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

6. （選用）安裝動畫套件 framer-motion，為版面增添未來感：

```
npm install framer-motion
```

步驟二：設定 Tailwind CSS

我們需要告訴 Tailwind 去哪裡尋找我們寫的 class。

1. 打開專案中的 tailwind.config.js，將內容修改為：

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

2. 打開 src/index.css，清空原本的內容，並加入以下 Tailwind 基本指令與自訂背景：

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  background-color: #0f172a; /* 深色沉穩背景 */
  color: #f8fafc;
```

步驟三：設計 Bento Grid 網格佈局

Bento Grid 的核心在於 CSS Grid。我們將在主程式中建立網格容器。

1. 打開 src/App.jsx，清空原本的代碼，替換成以下的架構：

```
import React from 'react';

function App() {
  return (
    <div className="min-h-screen bg-slate-900 p-8 flex items-center justify-center">
      {/* 這是 Bento Grid 的主容器 */}
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 max-w-6xl w-full auto-rows-[200px]">

        {/* 卡片 1 - 佔據較大版面 */}
        <div className="col-span-1 md:col-span-2 md:row-span-2 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700">
          <h2 className="text-3xl font-bold mb-4">主打視覺區</h2>
          <p className="text-slate-400">這裡是內容最豐富的區塊，適合放主要視覺、核心理念或大張圖表。</p>
        </div>

        {/* 卡片 2 - 寬扁型 */}
        <div className="col-span-1 md:col-span-2 bg-gradient-to-br from-indigo-500 to-purple-600 rounded-3xl p-6 shadow-lg flex items-end">
          <h2 className="text-2xl font-semibold text-white">高光色彩區</h2>
        </div>

        {/* 卡片 3 - 方型 */}
        <div className="col-span-1 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700 flex flex-col justify-center items-center">
          <h3 className="text-xl font-medium">數據 1</h3>
          <p className="text-3xl text-indigo-400 font-bold mt-2">+80%</p>
        </div>

        {/* 卡片 4 - 方型 */}
        <div className="col-span-1 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700 flex flex-col justify-center items-center">
          <h3 className="text-xl font-medium">數據 2</h3>
          <p className="text-3xl text-purple-400 font-bold mt-2">+120k</p>
        </div>

        {/* 卡片 5 - 橫跨三格 */}
        <div className="col-span-1 md:col-span-3 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700">
          <h3 className="text-xl font-medium mb-2">更多資訊</h3>
          <p className="text-slate-400">橫長型的區塊適合拿來做步驟說明、文章列表或是橫向捲動的圖集。</p>
        </div>

        {/* 卡片 6 - 小正方型 */}
        <div className="col-span-1 bg-gradient-to-tr from-pink-500 to-orange-400 rounded-3xl p-6 shadow-lg">
          <h3 className="text-xl font-medium text-white">加入我們</h3>
        </div>

      </div>
    </div>
  );
}

export default App;
```

🔗 網格原理解析：

- `grid-cols-4`: 讓畫面總共分成 4 個欄位。
- `auto-rows-[200px]`: 每一列的預設高度固定為 200px。
- `col-span-x`: 設定該卡片要橫向跨越多個欄位（例如 2 格）。
- `row-span-x`: 設定該卡片要垂直跨越多個列（例如 2 格）。

步驟四：加入微動畫（質感升級）

為了讓網站有「高級感」，我們可以利用剛剛安裝的 `framer-motion`。

1. 在 `App.jsx` 最上方引入 `motion`：

```
import { motion } from 'framer-motion';
```

2. 將你的 `<div className="col-span-1...">` 卡片替換成 `<motion.div>`，並加上 `whileHover` 與 `transition` 屬性，例如：

```
<motion.div
  whileHover={{ scale: 1.02 }}
  transition={{ type: "spring", stiffness: 300 }}
  className="col-span-1 md:col-span-2 md:row-span-2 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700"
>
  {/* 內容 */}
</motion.div>
```

💡 提示：你可以把每個卡片都加上 `motion.div`，這會讓滑鼠移過去時，卡片有輕微回彈放大的高級手感喔！

步驟五：啟動並預覽成果

在終端機中確保你在 `bento-grid-app` 資料夾內，執行以下指令啟動開發伺服器：

```
npm run dev
```

打開瀏覽器前往終端機顯示的網址（通常是 `http://localhost:5173`），你就會看到極具設計感的 **Bento Grid** 排版呈現在你眼前了！🔥

🔗 下一步挑戰？

1. 自訂排版：試著修改各個卡片的 `col-span` 和 `row-span` 的數字，玩出屬於你自己的版面。
2. 加入圖標：安裝 `lucide-react`，在卡片中放入漂亮的 Icons。
3. 響應式調整：在手機版時（`md`: 以外的狀態），觀察卡片是如何自動變成單欄排列的，並調整間距。

🌟 額外進階：引進 TradingView 專業看盤圖表與畫圖工具

如果你想讓這個網頁具備「專業看盤」的功能（包含畫線、拉區間、甚至看基本面與法人），完全不需要自己接 API 刻到頭破血流，我們可以直接使用現成的 **TradingView** 核心！

1. 安裝套件：在終端機輸入以下指令安裝

```
npm install react-ts-tradingview-widgets
```

2. 在你的頁面中使用圖表：你可以建立一個像是 `AnalysisView.jsx` 的新元件，把最夯的進階圖表放進去：

```
import React from 'react';
import { AdvancedRealTimeChart, CompanyProfile, SymbolInfo } from "react-ts-tradingview-widgets";

export default function AnalysisView() {
  return (
    <div className="flex flex-col gap-6 p-8">
      {/* 上方放置大圖表 (內建畫線工具、K線上萬種指標) */}
      <div className="h-[600px] rounded-2xl overflow-hidden border border-slate-700 shadow-xl">
        <AdvancedRealTimeChart symbol="NASDAQ:AAPL" theme="dark" autosize />
      </div>

      {/* 下方放置財報與基本分析 */}
      <div className="grid grid-cols-2 gap-6 h-[400px]">
        <div className="rounded-2xl overflow-hidden border border-slate-700">
          <CompanyProfile symbol="NASDAQ:AAPL" colorTheme="dark" width="100%" height="100%" />
        </div>
        <div className="rounded-2xl overflow-hidden border border-slate-700">
          <SymbolInfo symbol="NASDAQ:AAPL" colorTheme="dark" autosize />
        </div>
      </div>
    </div>
  );
}
```

這招能讓你的專案瞬間提升到「專業 FinTech 網站」的等級喔！

✎ 最終進階：加入全球測向儀與即時新聞牆

如果你希望你的網站不只有圖表，還能像 Bloomberg (彭博社) 一樣不斷跳出最新的華爾街即時新聞，以及判斷目前市場情緒的「技術分析儀表板」，你同樣可以使用 react-ts-tradingview-widgets 中的強大組件！

1. 引入套件中的 Timeline 與 TechnicalAnalysis：

```
import React from 'react';
import { Timeline, TechnicalAnalysis } from "react-ts-tradingview-widgets";

export default function MarketNewsView() {
  return (
    <div className="flex flex-col md:flex-row gap-6 p-8 h-screen bg-slate-900 border border-slate-700">

      {/* 區塊 1：動態市場買賣情緒指針 */}
      <div className="flex-1 rounded-2xl overflow-hidden bg-slate-800 p-4 relative">
        <h3 className="text-white font-bold mb-4 text-xl">S&P 500 市場情緒</h3>
        {/* 這個指針會根據真實技術指標自動告訴你現在是 Strong Buy 還是 Strong Sell */}
        <TechnicalAnalysis symbol="SPY" colorTheme="dark" width="100%" height={400} />
      </div>

      {/* 區塊 2：不斷更新的全球即時新聞牆 */}
      <div className="flex-1 rounded-2xl overflow-hidden bg-slate-800 p-4 h-[600px]">
        <h3 className="text-white font-bold mb-4 text-xl">即時滾動新聞</h3>
        <Timeline colorTheme="dark" feedMode="market" market="stock" height="100%" width="100%" />
      </div>
    </div>
  );
}
```

只要貼上這段程式碼，你的網站就能擁有會發亮的動態儀表板以及無窮無盡的即時財經新聞，讓作品的完整度破表！

💧 大師級密技：零後端！突破 5MB 限制的超流暢頭貼上傳器

在無伺服器 (Serverless) 的純前端架構中，我們通常會使用 localStorage 來永久記憶使用者的資料。但是，瀏覽器的本機空間通常有 **5MB** 的大小限制！如果使用者上傳了一張高畫質大頭貼 (Data URI Base64 編碼)，很可能會瞬間把容量塞爆導致整站崩潰。

解法：利用 HTML5 <canvas> 在前端瞬間無損壓縮！

這是一個超級專業的黑科技，我們可以在拿到使用者圖片的瞬間，在背景畫出一張 200x200 像素的隱形畫布，把大圖縮進去，再把它輸出。這會讓一張好幾 MB 的圖片瞬間縮小到只有大約 10KB，完美寫入 localStorage！

```
// 零後端：大頭貼自動壓縮器 (HTML5 Canvas 技巧)
const handleAvatarChange = (e) => {
  const file = e.target.files?.[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = (event) => {
    const img = new Image();
    img.onload = () => {
      // 1. 建立隱形畫布
      const canvas = document.createElement('canvas');
      const MAX_SIZE = 200; // 最寬或最高 200px

      let { width, height } = img;
      if (width > height) {
        if (width > MAX_SIZE) { height *= MAX_SIZE / width; width = MAX_SIZE; }
      } else {
        if (height > MAX_SIZE) { width *= MAX_SIZE / height; height = MAX_SIZE; }
      }

      // 2. 寫入畫布並等比例縮放
      canvas.width = width;
      canvas.height = height;
      const ctx = canvas.getContext('2d');
      ctx?.drawImage(img, 0, 0, width, height);

      // 3. 輸出成 Base64 字串，大功告成！直接可以存入 localStorage
      const tinyBase64Data = canvas.toDataURL('image/jpeg', 0.8);
      console.log("壓縮完成：", tinyBase64Data);
    };
    img.src = event.target?.result; // 觸發 img.onload
  };
  reader.readAsDataURL(file); // 讀取原始大檔
};
```

📱 終極進化：讓網頁變身超級手機 App (PWA)

這套系統不只在電腦的超薄邊框螢幕上很炫酷，它現在還是一台貨真價實的「跨平台應用程式」！我們使用了現代化的 **漸進式網頁應用程式 (PWA)** 架構，讓它擁有能夠像一般 App 一樣「掃描 QR Code 安裝到手機桌面」的神奇能力。

1. 手機專屬「無縫底部選單」架構

在傳統電腦版，我們設計了極具質感的「隱藏式側邊欄 (w-64)」；但是在手機上，這會完全吃光寶貴的空間。解法非常簡單直接：我們讓大螢幕版 (sm:flex) 保有左側面板；但在手機極限畫面上，我們實作了一條純黑玻璃質感的 **底部控制列 (Bottom Tab Bar)**，完全隱藏在 sm:hidden fixed bottom-0 屬性裡。只要用手機瀏覽，導航列會自動從左邊移到底部，擁有高階 iOS App 的沉浸視角。

2. QR Code 一鍵轉生 APP

當你在電腦瀏覽器中點擊右上角的「Share」圖標，會彈出我們精心手刻的 QR Code 視窗。這裡整合了強大的 qrcode.react 與 vite-plugin-pwa 插件：

- vite.config.ts 核心設定**：我們宣告了這個專案的 manifest，綁定了專案名稱如 QuantTrd、背景為深邃黑 (#0a0a0f)，並且讓它 display: 'standalone' —— 這代表當專案被安裝到手機時，最上方的醜陋瀏覽器網址列會被強制關閉消失！
- 網路共享**：我們打開了 Vite 的 host: true。這讓你只要跟電腦連著同一個 WiFi，拿出手機鏡頭一掃 QR 視窗，這套 QuantTrd 金融分析系統就會突破時空，直接跑到你的手掌心。

恭喜你！跟著這些技術，你已經將一份單純的切版作品，升級成整合「實質偏好記憶、無盡新聞瀑布流、跨螢幕無痛轉移、並支援本機影像壓縮演算」的 **生產力級別高階 Web APP 完全體**！

🚀 完美落地：Vercel 佈署與伺服器代理 (Proxy)

當你準備把作品推向全世界時，我們選擇了當今最強大的前端雲端平台 **Vercel** 來進行上架。但在這之中，有一個初學者必定會踩到的大坑：**CORS 跨域請求阻擋與 API Proxy 斷線問題**！

1. 為何本地會通，上線卻壞了？

我們在開發時，利用了 vite.config.ts 中的 proxy 將我們對 /api/finance 的請求，偷偷轉發給了 Yahoo 的股票主機。這在自己電腦上非常管用。但注意！當你將程式碼 Push 到 GitHub 並交給 Vercel 時，Vercel 只會把編譯完的靜態 HTML/JS 檔拿去渲染，它完全不知道你的 Vite 設定有這條代理規則！

2. 解法：撰寫 vercel.json 伺服器代理配置

為了解決這個問題，我們必須在專案的根目錄，直接命令 Vercel 的伺服器引擎去幫我們抓資料。我們新增了 vercel.json 檔案：

```
{
  "rewrites": [
    {
      "source": "/api/finance/:match*",
      "destination": "https://query1.finance.yahoo.com/:match*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

這個「黑魔術」會讓 Vercel 建立一個無伺服器 (Serverless) 路由。任何打向 /api/finance 的 API，Vercel 雲端主機都會充當白手套，替你向金融伺服器獲取資料，達成完美繞過所有 CORS 阻擋的神級技巧！

3. 客製化真實新聞分頁系統 (Pagination)

為了解決第三方元件 (如 TradingView) 的免費數量限制，我們在 Views.tsx 自己手刻了新聞處理器。我們把抓下來的龐大新聞陣列 (Array)，利用狀態機 useState 搭配超簡單的切片數學：

```
const currentNews = news.slice((page - 1) * pageSize, page * pageSize);
```

我們因此成功做出了擁有 1, 2, 3, 4, 5 實體翻页按鈕的超炫新聞牆。現在，你的系統不僅 UI 完美，連背後的資料處理邏輯也達到了業界級別的扎實度！

💡 最終神髓升級：無伺服器後端 (BaaS) 與 Excel 雲端聯動

如果一個網頁關掉後資料就消失，那只能稱作展品。我們在這裡導入了目前業界最潮的 **全端技術** 來讓我們的系統變成一個真正的「雲端產品 (SaaS)」！

1. 拋棄假存檔，迎向真正的資料庫 (Supabase)

我們已經完全拔掉了不靠譜的 LocalStorage，改用被譽為「開源版 Firebase」的最強利器：**Supabase**。

- 真正的登入系統 (Authentication)**：我們利用了 @supabase/supabase-js，輕鬆刻出了一個帶有點點觸控手感的 [Sign Up] 與 [Log In] 畫面。有了這個平台，系統不再是任人瀏覽的靜態網頁，而是只有擁有「認證身分 (JWT Token)」的會員才能進入的封閉生態圈。
- user_metadata 神級運用**：我們運用了非常取巧且高階的技術——我們不需要在 Postgres 裡建一堆複雜的資料表，而是直接把使用者的「總資產餘額」、「自選股代碼陣列 (Watchlist)」與「系統設定」壓縮成一個巨大的 JSON，這包 JSON 就直接掛在 Supabase 帳號的 user_metadata 屬性裡！這意味著不管你在手機、平板還是另一台電腦登入，你的 \$124,562 餘額、你剛剛偷偷加入清單的 BTC-USD，全都完美無縫地同步出現。

2. 專業級「雲端試算表 (Data Studio)」

我們聽見了專業操盤手的聲音：他們根本離不開 Excel。因此我們在側邊欄大膽加入了一個全新的 **Data Studio**。

- SheetJS (xlsx)**：我們引入了這支傳奇的開源套件，它能夠直接從你的電腦上把「二進位 (Binary)」的 .xlsx 或 .csv 檔案拆解。
- 雙向編輯**：檔案一載入，我們用 Tailwind 將它渲染成一張滿版的暗黑晶片風格資料表 (<table>)。最酷的是，每一個「儲存格 (td)」我們都塞進了無框線的 <input />。當你在網頁裡改了數字，按下 [Export] 按鈕，這張被修改過的表會直接「再次被打包成一個 Excel 檔案」下載回你的電腦本機！

🎯 給你的挑戰

這整個作品已經從一個切版作業，進化成了足以拿去跟全世界募資的完整商業級應用了。唯一的挑戰是：你能不能加上「加密貨幣支付」的按鈕？或者是把 Data Studio 的圖表直接用 Recharts 畫出來？天空才是你的極限，去創造吧！

🌟 最新進階：實作 Yahoo Finance 搜尋與優化 Watchlist 體驗

在這個專案的最後，我們將原本只是靜態介面的「Find Opportunities」升級成了真正會動的搜尋引擎，並大幅優化了右側 Watchlist 的操作手感！

1. 實作 Debounce 搜尋 (防抖動)

在 SearchWidget.tsx 中，如果使用者每打一個字我們就發送一次 API 請求，很可能會瞬間被 Yahoo Finance 的伺服器拉黑 (Rate Limit)。因此我們使用了 setTimeout 來實作「防抖動 (Debounce)」：

```
useEffect(() => {
  const search = async () => {
    // 串接 Yahoo Finance API...
  };

  // 使用者停止輸入 500ms 後才發送請求
  const timeoutId = setTimeout(search, 500);
  return () => clearTimeout(timeoutId); // 如果使用者還在打字，就清除上一次的計時器
}, [query]);
```

2. 優化 Watchlist 的操作體驗 (樂觀 UI)

原本的 Watchlist 在新增股票時，會「等待」Supabase 更新完成後才清空輸入框，這會讓使用者覺得網站卡卡的。我們將邏輯改為「樂觀 UI (Optimistic UI)」：使用者一按下新增，馬上清空輸入框並給予成功的回饋，然後在背景偷偷進行 Supabase 的同步。同時，如果因為 API 限制而抓不到資料，我們也修改了判斷邏輯，明確顯示「Failed to load data. This might be due to API rate limits」而非誤導人的「Watchlist is empty」。

3. 實作「雙引擎」API 備援機制 (Dual-Fetch Strategy)

在抓取股票資料時，如果短時間內發送太多次請求（例如 Watchlist 裡有 10 支股票就發送 10 次），很容易被 Yahoo Finance 暫時封鎖 IP (HTTP 429 Too Many Requests)。為了解決這個問題，我們實作了業界水準的雙重防護架構：

- 第一線防護 (Yahoo Batch API)**：我們捨棄了 Promise.all 併發請求的寫法，改用 Yahoo Finance 的 /v7/finance/quote?symbols=AAPL,TSLA... 批次查詢端點。如此一來，不管你的 Watchlist 有多少支股票，系統都只會發送 1 次 API 請求，大幅降低被封鎖的機率！
- 第二線防護 (Finnhub Fallback)**：如果第一線依然被封鎖，系統會自動切換到第二引擎。只要在環境變數 .env.local 放入免費註冊的 VITE_FINNHUB_API_KEY，系統就會在背景安靜地改向 Finnhub 索取最新的股價資料（甚至還內建了加密貨幣的代號轉換器，自動把 BTC-USD 轉換成 Finnhub 讀得懂的 BINANCE:BTCUSD）。

這兩個看似微小的改變，將會讓你的 Web App 從「能用」升級為「超級好用且永不斷線」的境界！

📌 期末專案大升級：將 Serverless 轉換為真正的 PHP 全端架構 (符合大學專題標準)

原本的架構雖然非常酷炫，但它是屬於「無伺服器 (Serverless)」或「後端即服務 (BaaS)」的架構 (React 前端直接跟 Supabase 資料庫對話)。為了符合學校期末作業「必須使用 PHP 作為後端」的要求，我們將專案進行了徹頭徹尾的全端分離大改造！

現在，這個專案是一個貨真價實的「前端 React + 後端 PHP + 資料庫 PostgreSQL」三層式架構。

1. 建立獨立的 PHP API 後端

我們在專案中建立了一個全新的 backend/ 資料夾，這代表我們自己寫了一個後端伺服器！

- composer.json**：我們導入了 firebase/php-jwt 套件，用來簽發和驗證使用者的登入 Token。
- config/db.php**：利用 PHP 的 PDO (PHP Data Objects) 模組，直接從後端透過 5432 Port 連線到 Supabase 提供的 PostgreSQL 資料庫。
- API 介面 (auth.php, profile.php, watchlist.php)**：我們完全自己手寫了註冊 (password_hash 密碼加密)、登入、讀取個資與增刪 Watchlist 的商業邏輯。

2. 拔除 Supabase JS，回歸最純粹的 Fetch API

在前端 React 方面，我們移除了原本依賴的 @supabase/supabase-js 套件，完全斷開前端與資料庫的直接連線（這是為了安全性，也是業界標準做法）。我們建立了一個 src/lib/api.ts 的「API 攔截器/封裝器」，它會：

- 自動從 LocalStorage 拿出 JWT Token。
- 幫你在每一次發送 fetch 請求給 PHP 時，自動塞入 Authorization: Bearer <token> 標頭。
- 統一處理 PHP 拋回來的 JSON 錯誤與網路例外。

3. 解決 Windows PHP 與 Supabase 連線的世紀大坑

在實作過程中，我們踩到了一個非常經典的地雷：SSL SYSCALL error: Connection reset by peer。

- 原因**：Supabase 的伺服器非常嚴格，要求用戶端在建立 TLS (HTTPS) 安全連線時，必須提供 SNI (Server Name Indication) 資訊。然而，Windows XAMPP 內建的 PHP pdo_pgsql 驅動程式較為老舊，發送憑證時常常會漏掉 SNI，導致 Supabase 覺得這是不明連線而瞬間切斷 (Reset by peer)。
- 解決方案**：我們使用了極為高階的解法！首先，將連接埠換成標準的 5432 來繞過學校的 6543 防火牆限制；接著，將 db.php 的安全模式從 sslmode=require 降級為 sslmode=prefer，完美避開了 Windows PHP 強制驗證 SNI 失敗而被踢下線的問題，同時又保有基本的加密能力！

🎓 結論

這份專案現在不只擁有流暢的 UI/UX 和酷炫的 React 動畫，它背後更有著扎實、符合業界規範的 PHP 伺服器與 JWT 驗證機制。這絕對是一份足以拿高分的期末專案作品！

🐳 最終上架：撰寫 Dockerfile 佈署至 Render

當我們把後端改寫為 PHP 後，如果要佈署到雲端平台 (如 Render)，最穩定的做法就是使用 **Docker** 來建立專屬的運行容器，確保 PostgreSQL 驅動程式完美運作！

1. 建立 Dockerfile

在專案的最外層目錄新增一個名為 Dockerfile (沒有副檔名) 的檔案，貼上以下內容：

```
FROM php:8.2-apache

# 安裝系統必備套件與 PostgreSQL 驅動程式
RUN apt-get update && apt-get install -y \
    libpq-dev \
    git \
    unzip \
    && docker-php-ext-install pdo pdo_pgsql pgsql \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# 開啟 Apache 的 mod_rewrite
RUN a2enmod rewrite

# 安裝 Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# 設定工作目錄
WORKDIR /var/www/html

# 僅複製 backend 資料夾到伺服器根目錄
COPY backend/ /var/www/html/

# 安裝 PHP 相依套件
RUN composer install --no-dev --optimize-autoloader

# 調整權限
RUN chown -R www-data:www-data /var/www/html \
    && chmod -R 755 /var/www/html

EXPOSE 80
```

2. 佈署至 Render

1. 將專案推送到 GitHub。
2. 進入 Render.com 建立新的 **Web Service**。
3. 選擇你的 Repository，在 Environment 選擇 Docker。
4. 點擊 Deploy！Render 就會幫你把 PHP API 完美運行起來了。

3. Vercel 前端設定

最後，前往 Vercel (你的 React 前端佈署地)：

- 在專案的 Settings -> Environment Variables 裡，新增：
 - Key: VITE_PHP_API_URL
 - Value: https://你的-render-網址.onrender.com/api
- 重新佈署 (Redeploy) Vercel 專案。

大功告成！你現在擁有了一個完全獨立、前後端分離、且搭載高安全性 JWT 認證的企業級全端專案！

進階升級：富途牛牛級別專業看盤與論壇系統

在專案的最後階段，我們將系統大幅升級，對標業界頂級金融軟體（如富途牛牛），加入了專業級的市場分析、沉浸式新聞與社群論壇。

1. 專業級 Analysis & Market 儀表板

我們引入了更多 react-ts-tradingview-widgets 的高階組件：

- **MarketsView**：加入了 MarketOverview (全球市場總覽) 與 StockHeatmap (S&P 500 板塊熱力圖)。
- **AnalysisView**：除了原有的進階圖表外，加入了 CompanyProfile (公司簡介)、SymbolInfo (個股資訊) 與 FundamentalData (基本面數據，包含 P/E, 市值等)。

2. 沉浸式 In-App 新聞閱讀器 (PHP Proxy Scraper) 升級版

為了解決點擊新聞會跳轉至外部網站的問題，我們實作了一支 PHP 爬蟲代理 (news_proxy.php)。

- **強化的爬蟲演算法**：針對 Yahoo Finance 最新改版的動態 DOM 結構，我們升級了 XPath 選擇器，能更精準地抓取 <article> 或 div.body yf- 內的文字，並過濾掉 SVG 與按鈕等雜訊。
- **前端呈現**：搭配 @tailwindcss/typography 插件，在網站內部彈出一個全螢幕的 Modal，實現「不出站」的沉浸式閱讀體驗。

3. Threads 風格投資論壇與 Cloudinary 雲端圖床

為了增加用戶粘著度，我們將原本的傳統論壇大幅翻新，打造出類似 Threads / Twitter (X) 的現代化社群體驗 (ForumView.tsx)。

- **瀑布流與選項標題**：發文不再強制需要填寫 Title，版面改為無接縫的對話流，提升隨興發文的意願。
- **Cloudinary 無後端圖床**：結合強大的 Cloudinary API，前端可以直接將使用者選取的圖片上傳至雲端，並自動轉為 Markdown 語法 ![image](url) 存入資料庫，無需修改後端儲存架構。
- **智慧解析標籤、提及與 Youtube 影片**：實作了 Regex 引擎，能自動抓取內文中的 # 關鍵字 與 @使用者名稱，將其高亮顯示為主要色彩，點擊標籤還能連動上方新增的搜尋列 (Search Bar) 即時過濾動態。如果貼上 YouTube 網址，系統還會自動將其轉換為內嵌的響應式播放器！
- **貼文編輯與管理**：現在你可以隨時點擊 Edit 按鈕，利用 PHP edit_post API 即時修改錯字或更新你的市場看法。
- **個人專屬主頁與自介**：點擊發文者的頭像，介面會平滑地切換至他的專屬主頁，展示他的「自我介紹 (Bio)」以及所有發布過的歷史貼文。

4. 系統級別的深淺色切換 (Dark & Light Mode)

為了讓網站能適應使用者的各種喜好，我們全面重構了 CSS 系統：

- **CSS Variables 重構**：在 index.css 定義了 :root (亮色) 與 .dark (暗色) 的色彩變數，如 --background, --card, --foreground，並在 tailwind.config.js 中對應。
- **動態主題切換**：在 Settings 面板中解鎖了主題切換功能，使用者切換時，系統會透過 React useEffect 動態地在 document.documentElement 切換 dark 樣式，實現真正意義上的深淺色無縫切換！

🚀 最新升級：財務預測儀表板、專業級 Data Studio 與 PWA 離線模式

在最新的更新中，我們進一步強化了系統的專業度與實用性，使其更接近專業金融終端機的水準。

1. 專業級 Data Studio (Excel 試算表檢視器)

我們將原本陽春的資料表格徹底重構，升級為具有專業質感的試算表介面 (SpreadsheetView.tsx)：

- **Excel 風格操作介面**：加入了類似 Excel 的 A, B, C 直行與 1, 2, 3 橫列標籤，並實作了藍色框線的「選取狀態 (Active Cell)」。
- **樣式工具列與公式列**：在頂部加入了模擬的文字樣式工具列 (粗體、斜體、對齊) 與功能強大的 $\times$ 公式/文字輸入列，讓你在網頁中也能享受原生的 Excel 編輯體驗。
- **無縫匯入與匯出**：結合 xlsx 套件，支援本地端 .xlsx 和 .csv 檔案的讀取與寫出，你的數據永遠掌握在自己手中。

2. PWA (漸進式網頁應用) 離線模式

為了解決使用者在沒有網路 (例如通勤中) 時無法查看數據的問題，我們全面導入了 PWA 離線快取機制：

- **Vite PWA 插件**：在 vite.config.ts 中配置了 vite-plugin-pwa，並啟用了 workbox 的 Aggressive Caching (積極快取) 策略，將所有的 HTML, CSS, JS 檔案與 Google 字體快取到瀏覽器。
- **無網路也能用**：在 main.tsx 中註冊了 Service Worker (virtual:pwa-register)。現在你可以點擊介面右上角的 Share 按鈕，將網站「加入主畫面」，它將像一個原生的手機 App 一樣在沒有網路的環境下啟動。

3. 華爾街共識與 EPS 預測儀表板 (Earnings Forecast Dashboard)

我們在 AnalysisView 中引入了一個全新的超強大組件 EarningsForecastWidget.tsx，幫助投資人預測公司未來的獲利能力，判斷是否值得買進：

- **Recharts 強大數據視覺化**：不依賴 TradingView，我們自行使用 recharts 打造了高質感的互動圖表。
- **年度財務預測 (Annual)**：使用長條圖 (BarChart) 與折線圖 (LineChart) 的組合，視覺化呈現歷史與未來的「總營收 (Revenue)」、「淨利 (Net Income)」以及最重要的「每股盈餘 (EPS)」。
- **單季成長趨勢 (Quarterly)**：使用漸層面積圖 (AreaChart) 追蹤單季營收，搭配折線圖顯示「營業利益率 (Operating Margin)」的變化，輕易抓出公司的成長動能。
- **獲利驅動力與風險分析 (Drivers & Risks)**：
 - 利用圓餅圖 (PieChart) 拆解公司的營收來源 (如硬體、服務、廣告)，分析主力業務。
 - 創新地採用雷達圖 (RadarChart) 來量化「不確定因素」與支出 (如研發費用、供應鏈風險、法規限制)，全面評估市場風險。

🏆 終極突破：建立 PHP 爬蟲代理 (Crumb Proxy) 串接 100% 真實財報

為了讓 EarningsForecastWidget 不再只是展示演算法模擬的資料，我們突破了 Yahoo Finance 最近極度嚴格的 API 封鎖機制 (Cookie & Crumb 驗證)，成功實現了無需付費訂閱即可獲取華爾街真實預測數據的神級功能！

1. 突破防爬蟲機制 (yahoo_forecast.php)

Yahoo Finance 封鎖了所有的直接 API 呼叫，要求開發者必須先取得合法的 Cookie 與稱為 crumb 的安全權杖。

我們在後端撰寫了一支聰明的 PHP 代理腳本：

- **Step 1**：透過 cURL 帶著瀏覽器的 User-Agent 模擬造訪首頁，偷偷把伺服器發給我們的 Set-Cookie 全部攔截並存起來。
- **Step 2**：帶著這些 Cookie 去敲門詢問 /v1/test/getcrumb，成功拿到這把一次性的安全鑰匙。
- **Step 3**：拿著鑰匙與 Cookie，正式向 /v10/finance/quoteSummary 請求 earningsTrend (未來預估)、incomeStatementHistory (歷史財報) 等五大核心模組，完美繞過所有防護！

2. 真實數據驅動的 React 圖表

我們把前端的圖表組件徹底改寫，拔除了所有的假數據生成器，改為：

```
const res = await fetch(`/api/yahoo_forecast.php?symbol=${symbol}`);
const data = await res.json();
```

現在，你看到的 2025(E) 預估營收、Forward P/E、甚至是前四年的真實歷史淨利，全部都是真正來自華爾街分析師的最新共識！這套系統已經從一個「漂亮的前端玩具」，正式蛻變成一個「機構等級的金融分析終端機」。

🏔 巔峰架構：導入 Python FastAPI 建立雙後端微服務 (Microservices)

隨著我們對金融數據的要求越來越高（例如即時量化運算、RSI 技術指標、突破 Yahoo Finance 的進階反爬蟲鎖定），單靠 PHP 處理所有事情已經不是業界的最佳實務。我們為專案導入了當前矽谷最主流的「雙後端微服務架構 (Microservices)」：

1. 職責分離：PHP 管會員，Python 管金融

- **PHP (網頁管理員)**：負責處理 auth.php 與 profile.php。使用 JWT 簽發憑證，守護會員登入、個人資料與帳戶餘額。
- **Python (量化運算大腦)**：我們在 backend/main.py 建立了一個輕量級且極速的 **FastAPI** 伺服器，專門處理即時股價 (/api/market/quote) 與分析師預估 (/api/market/forecast)。

在前端 src/lib/api.ts 中，我們定義了兩個不同的抓取函數：

```
const PHP_API_BASE = import.meta.env.VITE_PHP_API_URL;
const PYTHON_API_BASE = import.meta.env.VITE_PYTHON_API_URL;

// 預設打向 PHP (處理會員認證，帶 JWT Token)
export const apiFetch = async (endpoint, options) => { ... }

// 專門打向 Python (處理純粹的金融運算，極速回傳)
export const quantFetch = async (endpoint, options) => { ... }
```

這樣的前端架構完美實現了「分流」，不僅讓系統效能最大化，也大幅降低單一伺服器過載的風險！

2. Python yfinance 暴力突破台股智慧偵測

Yahoo Finance 最近的 Cookie/Crumb 阻擋機制讓全世界的 PHP 爬蟲哀鴻遍野，但 Python 的神級套件 yfinance 透過社群的持續更新，天生自帶了自動破解與輪替機制！我們不僅利用 yfinance 抓取真實的華爾街 EPS 預測，更實作了貼心的**台股智慧偵測 (Auto-Detect)**：

```
def yf_symbol(code: str, market: str):
    code = str(code).strip().upper()
    # 自動偵測：如果輸入的是 4 個數字 (如 2330, 2317)，自動補上 .TW 轉換為台股代碼
    if market == "台股" or (code.isdigit() and len(code) == 4):
        return code + ".TW"
    return code
```

現在，使用者在前端的 Watchlist 搜尋框只要直覺地輸入 2330，系統就會自動將其轉換為 2330.TW 並向 Python 發送請求，無縫獲取台股積電的即時盤中報價與漲跌幅！

3. 樂觀 UI (Optimistic UI) 帶來零延遲操作手感

在現代化的高階 App 中，如果按了「新增自選股」還要看著載入圈圈轉個 1 秒鐘，體驗就會大打折扣。我們在 Watchlist.tsx 中導入了極致的**樂觀 UI 更新 (Optimistic UI)** 技巧：

```
// 1. 使用者一按下新增，瞬間在前端畫面更新 (Optimistic Update)
const newWatchlist = [...watchlist, sym];
await updateProfile({ watchlist: newWatchlist });

// 2. 背景偷偷發送 PHP API 同步資料庫
try {
    await apiFetch('/watchlist.php', { method: 'POST', body: { symbol: sym } });
} catch (err) {
    // 如果 PHP 後端突然掛掉，依然不影響前端操作，只在 Console 發出警告
    console.warn("Failed to sync new symbol to PHP backend. It is only saved locally.");
}
```

透過先更新畫面、後同步伺服器的策略，我們徹底消滅了網路延遲 (Latency) 帶來的不適感，讓整套 Bento-Grid 金融平台的流暢度達到了宛如原生 iOS App 般的絲滑境界！🌀